

ConferenceCheck Mobile: Analytics, Meals and Notifications: Validation Report

Project: ConferenceCheck Mobile: Analytics, Meals and Notifications (component 4.3b) **Student:** Ndashi Bwalya Chisanga, computer number 2021470105 **Supervisor:** Mr. Mofya Phiri
Repository: <https://github.com/ndashi-chisanga6/ConferenceCheck-Mobile-Analytics-Meals-and-Notifications> **Validated state:** tag final-resubmission

This document reports the validation results and describes the path that produces them: the seeded dataset, the commands that reproduce every figure, and the acceptance criteria each run is held to (proposal Section 5, Table 3). Every number below is taken from a committed results file or from a named test or command output, and the source is named alongside it.

This version replaces the one at tag milestone-12-final-submission. It answers the supervisor's comments on that version: the per-criterion status replaces the earlier claim that every criterion was met, the concurrency result now comes from an experiment that can tell the design apart from a naive one, export correctness is checked against SQL rather than against another API endpoint, dashboard freshness is measured rather than derived, and access control and offline replay are covered (Sections 7 and 8). The work behind these changes is listed with its commits in docs/remaining_work.md.

Conventions. Latencies are quoted in milliseconds to one decimal place. Counts and rejection rates are exact. All timings are wall-clock, from named runs on a local PHP 8.5.8 / PostgreSQL 18.2 stack on 10 October 2026.

Contents

Section	Subject	Page
0	Headline results	2
0.1	Auditable evidence	3
1	Meal voucher redemption correctness	5
2	Session attendance and capacity correctness	7
3	Latency against the proposal's targets	7
4	Analytics dashboard freshness	8
5	Export correctness	9
6	Notification delivery	9
7	Access control	10
8	Offline replay	11

Section	Subject	Page
9	Baseline comparison, and its limits	12
10	Reproducibility	13
10.1	The environment these commands assume	13
10.2	Commands	13

0. Headline results

The evaluation exercise seeds a 500-attendee event with three meal categories, a voucher issued per attendee per category for 1,500 vouchers in all, and ten sessions (`tools/seed-evaluation-event.php`), then drives it through the real API rather than through unit-level mocks.

Criterion (proposal Table 3)	Target	Result	Status
Duplicate redemption rejection, including concurrent submissions	100%	100/100 sequential re-scans refused; on PostgreSQL behind five server processes, 50/50 five-way races with exactly one redemption with the lock, the constraint or both in place; with both removed, 48/50 vouchers redeemed more than once (Section 1)	Met
Scan validation latency	median < 500 ms, p95 < 1 s	voucher scan 145.3 ms / 261.0 ms (n=200); session scan 128.4 ms / 228.7 ms (n=40)	Met on loopback only (Section 3)
Dashboard freshness	≤ 30 s behind the database	20 injected check-ins: median 18.1 s, p95 28.4 s, max 28.5 s, 20/20 within 30 s	Met in the measured run; by design the worst case is one 30 s cycle plus a fetch, fractionally over 30 s (Section 4)
Capacity warning before 100% occupancy	warning delivered before full	two automated transition tests; one observation on an emulator	Partly met: shown, not timed
Push delivery	≥ 95% of online devices	one device, one observed delivery of about 5 s	Not measured

Criterion (proposal Table 3)	Target	Result	Status
	within 30 s		
In-app inbox reach	100% of recipients on next open	a recipient record for every recipient, inbox reads filtered to recipients and tested; not measured on devices	Partly met
Export correctness	row counts and totals equal to the database	all four exports equal to direct SQL on row counts and on a total each (Section 5)	Met
Comparison with a manual baseline	measured against a staffed paper line	not run (Section 9)	Not run

Four criteria are met, one of them on loopback only; two are partly met; push delivery across a fleet was not measured; and the manual baseline was not run. Two further results test the guarantees the design rests on: all three categories were redeemable independently for 20 of 20 attendees, and 20 of 20 sequential duplicate session scans were refused.

Sources: `tools/evaluation-results.json` (python `tools/run-evaluation.py` against `tools/seed-evaluation-event.php`), `tools/concurrency-results.json` (python `tools/run-concurrency-experiment.py`) and `tools/freshness-results.json` (python `tools/measure-dashboard-freshness.py`).

0.1 Auditable evidence

Validated state. Tag `final-resubmission`. The exact commit it resolves to is verifiable with:

```
git rev-list -n1 final-resubmission
```

(`final-resubmission` is an annotated tag; a bare `git rev-parse` on it returns the tag object's own hash, not the commit, so use `rev-list -n1` or `rev-parse final-resubmission^{commit}` to resolve to the commit.)

Test suite. Command and output, run on PHP 8.5.8 with PostgreSQL 18.2 on 10 October 2026:

```
$ php artisan test
{"tool":"phpunit","result":"passed","tests":70,"passed":70,"assertions":288}
```

Of those 70 tests, 31 (152 assertions) are this component's, all in `tests/Feature/ConferenceApiTest.php`. The other 39 are the Laravel starter kit's example, dashboard, authentication and settings tests, which came with the application scaffold and test

none of this component's behaviour. Twelve of the 39 render pages and need the frontend built (npm run build) before they pass. The 31 component tests are:

- authentication and roles: test_login_returns_token, test_register_cannot_pick_organiser_role
- analytics: test_analytics_summary_endpoint
- meal vouchers: test_valid_meal_voucher_scan, test_duplicate_meal_voucher_scan_rejected, test_constraint_rejection_returns_409_not_500, test_invalid_meal_voucher_scan_rejected, test_redeeming_one_category_leaves_the_attendees_other_categories_redeemable, test_redemptions_cannot_be_edited_or_deleted_away
- sessions and check-in: test_valid_session_attendance_scan, test_duplicate_session_attendance_scan_rejected, test_session_scan_that_loses_a_race_gets_409_not_500, test_check_in_that_loses_a_race_is_a_duplicate_not_a_500, test_session_scan_records_the_scanning_device, test_capacity_threshold_transitions_alert_organisers
- notifications: test_notification_send_in_demo_mode, test_missing_firebase_config_fails_instead_of_faking_success, test_custom_target_is_rejected_instead_of_sent_to_everyone, test_pushes_go_out_after_the_recipient_rows_are_committed, test_notification_send_uses_fcm_v1_when_configured, test_delivery_is_recorded_against_each_recipient_individually
- reports: test_csv_report_endpoint_returns_downloadable_csv, test_every_export_has_one_row_per_database_record
- access control: test_attendee_can_fetch_own_qr_token, test_my_attendee_returns_404_when_no_record_is_linked, test_attendee_cannot_read_staff_or_organiser_routes, test_attendee_can_only_open_their_own_voucher, test_notifications_are_only_readable_by_their_recipients, test_only_the_owner_can_delete_a_device_token, test_linking_an_attendee_record_does_not_demote_staff, test_scanner_cannot_download_reports

PHPStan level 7: clean. Pint: clean. Flutter: flutter analyze no issues, flutter test 27/27 passed.

Re-verification, 10 October 2026. Every command in Section 10 was run on that date against the code at the validated state. The evaluation seed produced 500 attendees, 3 meal categories, 1,500 vouchers and 10 sessions; the evaluation round reported 200 of 200 scans accepted, 100 of 100 sequential duplicates refused, 20 of 20 single-server races with exactly one success, 20 of 20 attendees redeeming all three categories independently, 20 of 20 sequential duplicate session scans refused, and all four exports equal to direct SQL. The concurrency experiment and the freshness measurement were run the same day; their results are in Sections 1 and 4.

Route coverage, and which routes are this component's. The backend exposes 47 API routes (php artisan route:list --path=api), distributed as: MealController 10, AttendeeController 8, SessionController 7, NotificationController 5, EventController 5, AnalyticsController 4, ReportController 4, AuthController 4.

The repository is a standalone Laravel 13 application, started from the Laravel starter kit, that implements the API contract agreed with component 4.3a. It is not a shared backend, and component 4.3a's own API is not part of it. Two of the eight AttendeeController routes, POST /events/{event}/attendees/check-in/scan and POST /events/{event}/attendees/{attendee}/check-in, are a minimal check-in stub written for this component, as agreed, so that it can be built and tested on its own. Check-in capture is component 4.3a's work; this component only reads the check-in data, for analytics. Integration with component 4.3a's own API was not demonstrated. AuthController and EventController are the authentication and event scaffolding both components need. The routes this component contributes are the meal voucher, session attendance, notification, analytics and reporting groups, plus GET /events/{event}/attendees/me, which lets the attendee QR screen fetch a real server-issued token.

The boundary is visible in the mobile codebase, which is this component's alone: mobile/lib/features/ contains dashboard, meals, sessions, notifications, reports, attendee, events, auth, profile and sync, and no check-in capture module. The only mobile code that mentions check-in at all is in features/dashboard/, where check-in counts are read back from the analytics endpoints.

1. Meal voucher redemption correctness

How a scan is decided. MealController::scanVoucher (app/Http/Controllers/Api/MealController.php) runs the redemption in one database transaction. It selects the voucher by its token with lockForUpdate(), so on PostgreSQL a second scan of the same voucher waits for the first transaction to finish. It then checks, in order, that the voucher belongs to the event (404 otherwise), that its status is unused (409 otherwise, with the status and the time of the earlier redemption), that the category is active and that the scan is inside the redemption window (422 otherwise). Only then does it mark the voucher redeemed and insert the meal_redemptions row, whose meal_voucher_id is unique. If that insert violates the unique constraint, the violation is caught outside the transaction and returned as the same 409. With the lock in place, a losing scan is refused by the status check after it has waited for the winner; the unique constraint is the backstop if the lock is missing.

Sequential re-scans: 100 of 100 refused after the first successful redemption.

Concurrent races, single server. In tools/run-evaluation.py, 20 of 20 races of five simultaneous scans resolved to exactly one success. The development server used for that run executes one request at a time, so this result cannot tell the design apart from a plain check-then-insert, and it is not the evidence for the concurrency claim. That evidence is the experiment below.

Concurrent races on PostgreSQL behind five server processes. `tools/run-concurrency-experiment.py` runs five separate PHP server processes against PostgreSQL and fires each race's five scans of one fresh voucher at the same instant, one to each process, so the requests overlap in the database. It runs every combination of the two safeguards, row lock on or off and unique constraint on or off, with the status check present in all of them. Each configuration is served from a scratch git worktree of the same commit, with `lockForUpdate()` deleted where the lock is off, and the constraint is dropped on a scratch database. The grid is run twice: as the code stands, and with a 50 ms pause injected between the status check and the write in every configuration, which holds the window in which two scans can both pass the check open on purpose. Fifty races per configuration:

Configuration	Races with exactly one success	Vouchers redeemed more than once	Responses
lock on, constraint on (as built)	50/50	0	50 × 200, 200 × 409
lock off, constraint on	50/50	0	50 × 200, 200 × 409
lock on, constraint off	50/50	0	50 × 200, 200 × 409
lock off, constraint off	2/50	48	190 × 200, 60 × 409
lock on, constraint on, 50 ms window	50/50	0	50 × 200, 200 × 409
lock off, constraint on, 50 ms window	50/50	0	50 × 200, 200 × 409
lock on, constraint off, 50 ms window	50/50	0	50 × 200, 200 × 409
lock off, constraint off, 50 ms window	0/50	50	236 × 200, 14 × 409

The naive configuration fails even without the injected pause, so the race is real under this load, and the test is able to detect it. With either safeguard in place there was exactly one redemption in every race, and no response was a 500. Each safeguard is therefore sufficient on its own, and the system as built carries both. What this does not measure is throughput: it tests correctness under contention, not how many scanners the system can serve at once.

Source: `tools/concurrency-results.json`.

Cross-category independence: 20 of 20 reserved attendees, each holding a voucher in all three categories, redeemed every category successfully and were then refused on a repeat of the first. Redeeming breakfast consumes breakfast and nothing else. The same guarantee is pinned at unit level by `test_redeeming_one_category_leaves_the_attendees_other_categories_redeemable`.

Ground truth: 280 redemption rows for 200 batch scans, 20 race winners and 60 cross-category redemptions, equal to a direct SQL count (Section 5).

The redemption record. A redemption cannot be changed or removed once it is recorded: the model refuses updates and deletes, the foreign keys from meal_redemptions restrict deletes rather than cascading them, and the API answers 409 instead of deleting an event, attendee or meal category that has redemptions. This is pinned by test_redemptions_cannot_be_edited_or_deleted_away.

Source: sequential_duplicate_rejection, concurrent_duplicate_rejection and cross_category_independence in tools/evaluation-results.json.

2. Session attendance and capacity correctness

The unique (session, attendee) constraint prevents double counting; 20 of 20 sequential duplicate session check-ins were refused in the evaluation run.

SessionController::scan locks the session row inside its transaction, which serialises scans of one session, so the duplicate check, the capacity count and the capacity transition are all computed under the lock. A scan that still reaches the unique constraint returns 409, not 500. That path is tested by test_session_scan_that_loses_a_race_gets_409_not_500, which inserts a competing row at the moment between the check and the insert. The session path was not part of the parallel experiment in Section 1, so it has not been tested under real parallel load.

Capacity threshold transitions (the configurable warning fraction, default 90%, and over-capacity) push an automatic organiser notification, sent after the attendance transaction commits. This is covered by test_capacity_threshold_transitions_alert_organisers and was observed once on an emulator (docs/evidence-capacity-alerts.png). It was not timed, so the criterion that the warning arrives before the session is full is shown, not measured.

Source: session_duplicate_rejection in tools/evaluation-results.json.

3. Latency against the proposal's targets

Nielsen's response-time thresholds, cited in the proposal's literature review, set the sub-second budget for scan-validation round trips at a catering service point where queue throughput matters.

Endpoint	n	Mean	Median	p95	Max	Target (median / p95)
Meal voucher scan	200	164.0 ms	145.3 ms	261.0 ms	316.6 ms	< 500 ms / < 1,000 ms
Session scan	40	143.3 ms	128.4 ms	228.7 ms	242.1 ms	< 500 ms / < 1,000 ms

Endpoint	n	Mean	Median	p95	Max	Target (median / p95)
Analytics summary	20	124.0 ms	117.9 ms	138.7 ms	208.2 ms	part of each dashboard refresh (Section 4)

These were measured over loopback on one development machine. The proposal's targets were written for venue Wi-Fi, which this run does not exercise.

The envelope these numbers hold in. These figures are one named run and a property of that run rather than of the system. Four runs of the evaluation script have been recorded on this machine: voucher-scan medians of 114.0 ms, 199.0 ms, 187.5 ms and 145.3 ms, with 95th percentiles of 210.5 ms, 304.3 ms, 286.7 ms and 261.0 ms. The spread is machine load, not code. On a development machine the voucher-scan median falls between roughly 110 ms and 200 ms and the 95th percentile between roughly 210 ms and 305 ms, inside the 500 ms and 1,000 ms targets in every run observed. Behaviour on venue Wi-Fi, behind a production server, or above 500 attendees is not established.

Source: `voucher_scan_latency`, `session_scan_latency` and `analytics_summary_latency` in `tools/evaluation-results.json`.

4. Analytics dashboard freshness

The dashboard polls rather than streams (a documented trade-off against a WebSocket layer, proposal Section 8). The app fetches the four analytics endpoints in parallel and schedules its next refresh 30 seconds after the previous one started (`mobile/lib/features/dashboard/application/analytics_providers.dart`).

Freshness was measured the way the proposal describes, by injecting check-ins and timing their appearance. `tools/measure-dashboard-freshness.py` runs a poller that reproduces the app's refresh exactly, injects 20 check-ins one at a time at random moments through the check-in API, and times each from the start of the injecting request to the end of the first dashboard fetch that shows it.

Trials	Median	p95	Max	Within 30 s
20	18.1 s	28.4 s	28.5 s	20/20

This measures the data path to the app, polling plus API; it does not include the time Flutter takes to repaint. By design the worst case is one full 30 s cycle plus the duration of a fetch, so a check-in that lands just after a fetch starts can appear fractionally later than 30 s. None of the 20 random trials did, but the design does not guarantee a strict 30 s ceiling.

Source: `tools/freshness-results.json`.

5. Export correctness

Each of the four CSV exports was downloaded through the API and compared with SQL run directly on the database, on its row count and on one total per export.

Export	Check	CSV	SQL	Match
attendance.csv	rows	500	500	yes
attendance.csv	attendees checked in	40	40	yes
meals.csv	rows	280	280	yes
meals.csv	redemptions per category	94 / 93 / 93	94 / 93 / 93	yes
sessions.csv	rows	10	10	yes
sessions.csv	attendance per session	40, then 0 for the other nine	same	yes
notifications.csv	rows	0	0	yes
notifications.csv	total recipients	0	0	yes

The evaluation event has no notifications, so the two `notifications.csv` checks compare zero with zero and show only that the export is well formed. A notification was not sent during the run because the local configuration uses live Firebase credentials. That export's row count is tested on seeded data, which does contain a notification, by `test_every_export_has_one_row_per_database_record`.

The exports read their rows in chunks of 1,000 while the file is being written (`lazy()` in `app/Http/Controllers/Api/ReportController.php`), so the whole export is not held in memory at once.

Source: `export_correctness` in `tools/evaluation-results.json`.

6. Notification delivery

Delivery is recorded per recipient. A recipient is sent only when one of that recipient's own device tokens was accepted, failed with a reason when all of theirs were refused, and left pending when they have no registered device and will receive the message through the in-app inbox instead. Before this was fixed, one aggregate outcome was applied to every recipient. The behaviour is pinned by

`test_delivery_is_recorded_against_each_recipient_individually`, which fails against the earlier implementation. Recipient rows are committed before anything is pushed, and the Firebase calls run outside the database transaction (`test_pushes_go_out_after_the_recipient_rows_are_committed`).

Demo mode is never counted as delivery. Firebase demo mode is off unless it is set explicitly. When it is on, nothing is pushed, a warning is logged, the response carries "demo": true, and the notification and every recipient are recorded as demo, never sent. A missing Firebase configuration with demo mode off is a failed send, not a silent demo (test_missing_firebase_config_fails_instead_of_faking_success).

Live delivery, as single observations. Live Firebase Cloud Messaging v1 delivery was observed end to end once on an Android emulator: an organiser send reached the device's notification shade (docs/evidence-push-delivered.png). Receipt took about 5 seconds on that one device. No run log of fleet delivery was committed, so the proposal's target of 95% of online devices within 30 seconds is not measured.

Inbox. Every notification is stored with a recipient record for each person it was sent to, and the in-app inbox shows a user only the notifications they were a recipient of (Section 7). This is tested; it was not measured on devices.

7. Access control

The supervisor found that an authenticated attendee assigned to the event could read another attendee's voucher token, the attendance export, the event's redemption history and a notification addressed to organisers, on a local database with synthetic data. Each was fixed and has a feature test that expects 403 and that fails against the earlier code:

Defect	Fix	Test
Attendee could open any voucher by id	an attendee can open only their own voucher; organisers and scanners can open any	test_attendee_can_only_open_their_own_voucher
Attendee could download the CSV reports and read the redemption history, the	reports are organiser only; the other four are staff (organiser and scanner) only	test_attendee_cannot_read_staff_or_organiser_routes, test_scanner_cannot_download_reports

Defect	Fix	Test
attendee list, the voucher list and session attendance		
Attendee could list every notification and open organiser-only ones	the list holds only notifications the user received, and opening any other returns 403; only organisers see recipient rows	test_notifications_are_only_readable_by_their_recipients

Scanners keep read access to the redemption history, because the scanner app's history tab uses it.

Four further defects of the same kind were found while fixing these and are covered the same way: registration let a caller choose the organiser role (test_register_cannot_pick_organiser_role); any organiser could delete any user's device token (test_only_the_owner_can_delete_a_device_token); the custom notification target passed validation and was sent to every attendee (test_custom_target_is_rejected_instead_of_sent_to_everyone); and linking an attendee record to a scanner demoted them to attendee (test_linking_an_attendee_record_does_not_demote_staff).

8. Offline replay

Scans made without connectivity are queued on the device and replayed later (mobile/lib/features/sync/application/scan_sync_controller.dart). Two defects in the replay path were fixed:

- **Recoverable failures were dropped.** Replay treated every HTTP error as a settled result, so an expired login or a server fault silently lost the scan. Now only a business answer settles a queued scan: 404 (invalid token), 409 (already redeemed or duplicate) and 422

(not redeemable now). Anything else, including no connection, 401, 403, 429 and 5xx, keeps that scan and every scan after it, and stops the run.

- **A concurrent enqueue could be overwritten.** Replay wrote the leftover list back over the whole queue, so a scan queued while a replay was running was wiped. Replay now removes only the scans it handled.

Tests in `mobile/test/scan_replay_test.dart` and `mobile/test/scan_replay_restart_test.dart` drive the replay itself with a fake API:

Test	What it shows
only business answers settle a replay	404, 409 and 422 settle; no connection, 401, 403, 408, 429, 500, 502 and 503 do not
an expired login keeps the scan and everything after it	a 401 mid-queue keeps that scan and the rest
a server fault keeps the scan for the next sync	a 500 keeps the queue
a scan queued while a replay is running is not lost	a scan added mid-replay survives
a scan queued during a failing replay stays behind the kept ones	queue order is preserved
already redeemed and invalid tokens are dropped	business answers clear the queue
scans queued before a restart are replayed after it	a queue left by an earlier app run, meal and session scans, is replayed
a scan kept after an auth failure survives a restart and syncs later	a 401 survives a restart and syncs on the next run

These run against a fake API in the test harness; replay was not exercised on a device. Camera decoding of QR codes was checked by hand and not measured: `mobile/test/qr_parser_test.dart` tests only the trimming of a manually entered token.

9. Baseline comparison, and its limits

The proposal's evaluation plan calls for comparing the voucher module against a simulated, human-staffed paper-register meal line. That baseline was **not** conducted: organising a staffed comparison exercise was not feasible within the project's scope. This is stated plainly rather than approximated, because a fabricated baseline number would be worse than admitting the gap.

What the evaluation does establish is the system-side bound: a median scan-to-decision time of 145.3 ms, with single-use enforcement that a paper register cannot provide at any speed (a photographed or reused paper voucher has no server-side check to fail).

10. Reproducibility

10.1 The environment these commands assume

PHP 8.5.8 for Windows, with the `pdo_pgsql` and `pgsql` extensions enabled in its `php.ini`, along with `openssl`, `curl`, `mbstring`, `zip`, `gd` and `fileinfo`. The PostgreSQL client library `libpq.dll` ships beside the executable. A PHP without those extensions loaded cannot connect to the database and none of the commands below will run.

The repository carries `php-pgsql.bat`, a one-line wrapper that resolves to a PHP under a `.tools` directory beside the repository; substituting `.\php-pgsql.bat` for `php` makes every command below work when PHP is not on `PATH`. The evaluation scripts find that interpreter themselves, or use the one named in the PHP environment variable. The interpreter itself is not committed, because a platform-specific binary does not belong in a source repository, but its version and required extensions are recorded here.

The rest of the toolchain, at the versions used: PostgreSQL 18.2, Python 3.14.0, pandoc 3.9.0.1, and Microsoft Word 16.0, which `tools/paginate_report.py` drives to export the PDFs and to measure the page numbers printed in the contents list. The Python packages the document build imports are pinned in `tools/requirements-docs.txt`. The evaluation scripts use only the Python standard library.

10.2 Commands

The evaluation round, against a local backend:

```
php artisan serve
php artisan tinker tools/seed-evaluation-event.php
python tools/run-evaluation.py
python tools/measure-dashboard-freshness.py
```

The concurrency experiment, which needs PostgreSQL and creates and drops its own scratch database, so no server needs to be running:

```
python tools/run-concurrency-experiment.py
```

These regenerate `tools/evaluation-results.json`, `tools/freshness-results.json` and `tools/concurrency-results.json`. They overwrite the recorded runs, so the figures quoted in this report come from the named runs committed in those files.

Backend test suite:

```
php artisan test
```

Static analysis:

```
php vendor/bin/phpstan analyse --memory-limit=1G
php vendor/bin/pint --test
```

PHPStan analyses in parallel worker processes that each receive PHP's `memory_limit`, 128 MB by default, which a worker on this project can exhaust; `--memory-limit=1G` makes the run deterministic. Whenever the analysis completes it reports zero errors at level 7.

Mobile:

```
flutter analyze
flutter test
```

Word and PDF deliverables, rebuilt from this markdown and from the report:

```
python tools/build_docx_deliverables.py
python tools/paginate_report.py
```

`tools/paginate_report.py --check` fails if the page numbers printed in the contents lists no longer match the rendered documents.